Aplicações de Processamento Digital de Sinais – 4456S-04

Prática 6: Filtragem de Imagem

Antônio Vinicius Corrêa dos Santos

28/09/2026

Imagem: `lena_c.jpg`, 256×256, convertida para escala de cinza com `rgb2gray` e normalizada para $[0, 1]$. Todas as figuras são geradas por `main.m` e gravadas em `plots/`.

1 Espectro de Fourier da imagem

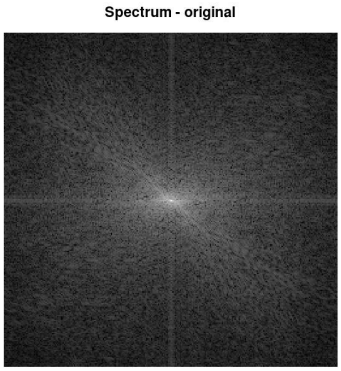


Figura 1: Espectro de magnitude, $\log(1 + |\mathcal{F}\{x\}|)$, centralizado com `fftshift`.

A energia se concentra no centro (baixas frequências) e cai rapidamente com o raio. As duas faixas claras horizontal e vertical vêm da descontinuidade entre as bordas opostas da imagem, que a DFT trata como periódica.

2 Parte I — filtragem no domínio espacial

Cada máscara foi aplicada com `filter2` e o resultado exibido ao lado da imagem original. A máscara passa-baixas foi normalizada por 1/9 para manter o ganho DC unitário. As saídas dos filtros derivativos são exibidas com `imshow(Y,[])`, porque têm valores negativos.

Tabela 1: Máscaras utilizadas e efeito observado.

Máscara	**Tipo**	**Efeito observado**
Passa-baixas	média 3×3	Suavização leve; bordas e textura do cabelo perdem definição.
Passa-altas	soma nula, centro 8	Só as bordas sobrevivem; as regiões de intensidade constante vão a zero.
Prewitt vert. / Sobel vert.	derivada em x	Realçam bordas verticais; Sobel dá contornos mais fortes por causa do peso 2 na linha central.
Prewitt hor. / Sobel hor.	derivada em y	O mesmo para bordas horizontais (chapéu, ombro).
Kirsch	direcional	Realce direcional de borda, com aspecto de relevo (*embossing*).
Laplaciano	segunda derivada	Detecta borda nas duas direções, mas a resposta é bem mais fraca que a dos de primeira derivada: só as transições mais abruptas aparecem, como linhas finas sobre fundo uniforme.

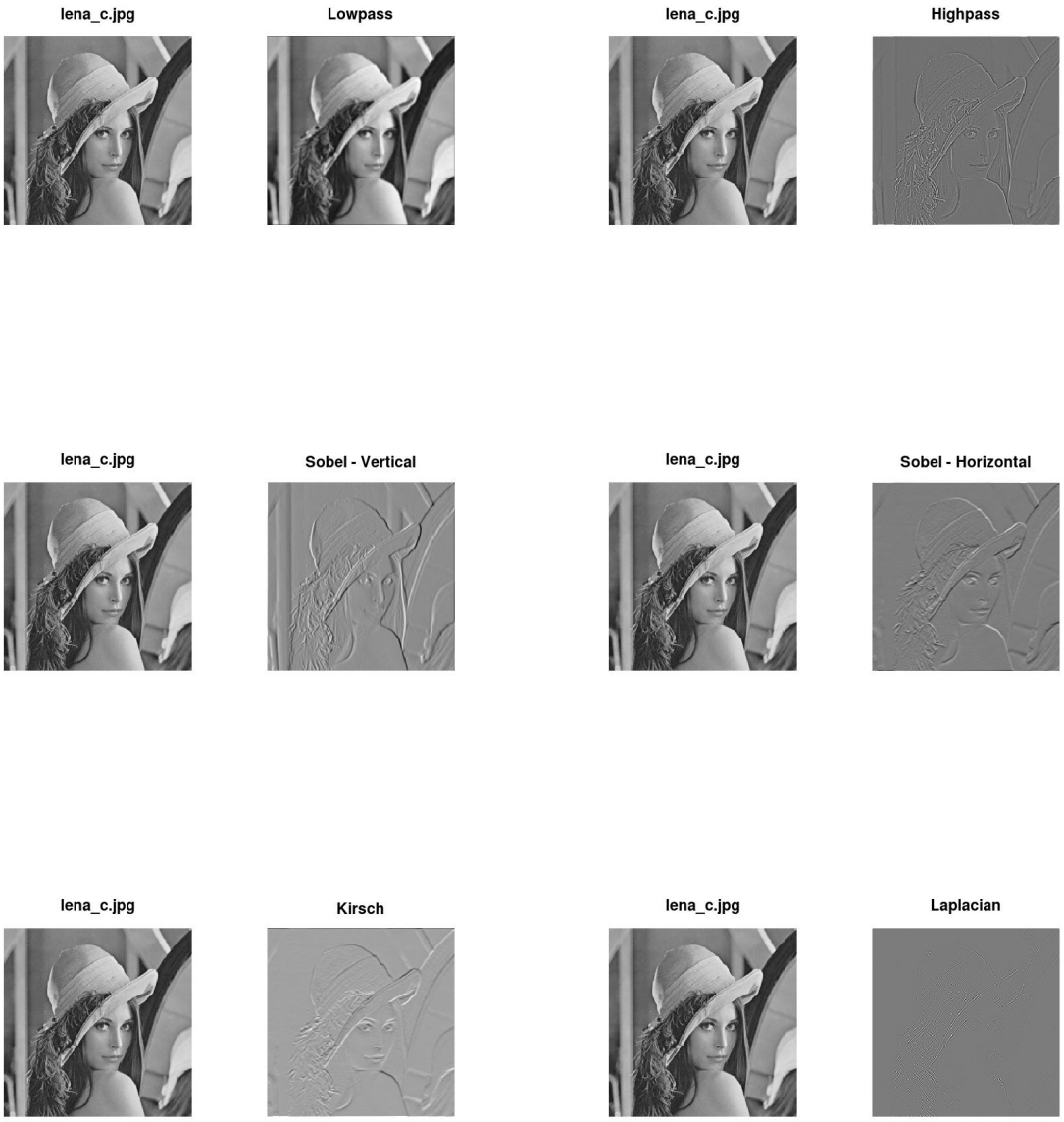


Figura 2: Original × filtrada: passa-baixas, passa-altas, Sobel vertical, Sobel horizontal, Kirsch e Laplaciano. As saídas de Prewitt estão em `plots/lena_c_Prewitt_*.jpg` e são visualmente próximas das de Sobel.

3 Parte II — filtragem no domínio da frequência

As máscaras são construídas sobre a matriz de distância D centrada, então o produto é feito com o espectro deslocado e desfeito antes da inversa:

$$y = \Re\left\{\mathcal{F}^{-1}\left[\text{ifftshift}\big(\text{fftshift}(\mathcal{F}\{x\}) \cdot H\big)\right]\right\}.$$

Multiplicar H centrada diretamente por `fft2(img)` produz uma imagem embaralhada; este foi o único ponto que exigiu atenção na implementação.

$$H_{\text{ideal}}(u,v) = \begin{cases} 1, & D \leq D_0 \\ 0, & D > D_0 \end{cases} \qquad H_{\text{butt}}(u,v) = \frac{1}{1+(D/D_0)^{2n}} \qquad H_{\text{PA}} = 1 - H_{\text{butt}}$$



Figura 3: Máscaras para $D_0 = 50$, $n = 2$.



Figura 4: Filtragem para $(D_0, n) = (50, 2)$, $(10, 2)$, $(120, 2)$ e $(50, 8)$. Em cada figura: original, passa-baixas ideal, passa-baixas de Butterworth e passa-altas de Butterworth.

Tabela 2: Fração da energia do espectro que atravessa cada máscara, $\sum |X \cdot H|^2 / \sum |X|^2$.

D_0	n	**PB ideal**	**PB Butterworth**	**PA Butterworth**
50	2	0,9942	0,9910	0,0043
10	2	0,9650	0,9540	0,0295
120	2	0,9996	0,9980	0,0004
50	8	0,9942	0,9936	0,0053

Observações:

- **Ideal × Butterworth.** O corte abrupto do filtro ideal produz *ringing*: ondulações concêntricas em torno das bordas, bem visíveis em $D_0 = 10$. É o efeito de Gibbs em duas dimensões — o equivalente espacial de H_{ideal} é uma função tipo *sinc* de suporte infinito. O Butterworth, com transição suave, não apresenta esse artefato; sua saída é apenas borrada.
- **Frequência de corte.** $D_0 = 10$ borra fortemente (resta pouco mais que a silhueta), $D_0 = 50$ suaviza a textura mantendo a forma, e $D_0 = 120$ quase não altera a imagem — com 256×256, o raio de Nyquist é 128, então $D_0 = 120$ deixa passar praticamente todo o espectro.
- **Ordem.** Aumentar n de 2 para 8 com D_0 fixo aproxima o Butterworth do filtro ideal: a energia retida sobe de 0,9910 para 0,9936 e começa a aparecer um leve *ringing*, ainda muito menor que o do corte abrupto.

- **Passa-altas.** $1 - H_{\text{butt}}$ retém menos de 3 % da energia e devolve apenas o mapa de bordas, coerente com o resultado das máscaras espaciais da Parte I.

4 Imagem com *blur* horizontal

O *blur* foi gerado com a máscara $h = \frac{1}{16}[1\ 1\ \cdots\ 1]$ (1×16).



Figura 5: Imagem borrada na direção horizontal e seu espectro. As faixas escuras verticais são os zeros do *sinc* da média móvel.

O espectro mostra o que o *blur* faz: a média móvel de 16 pontos tem resposta $\sin(8\omega)/(16\sin(\omega/2))$, cujos zeros anulam completamente linhas inteiras de frequências horizontais. Essa informação é perdida, não apenas atenuada.

Tabela 3: Energia retida ao filtrar a imagem já borrada.

D_0	n	**PB ideal**	**PB Butterworth**	**PA Butterworth**
50	2	0,9997	0,9994	0,0002
10	2	0,9946	0,9860	0,0045
120	2	1,0000	0,9999	0,0000
50	8	0,9997	0,9997	0,0002

Comparando com a tabela anterior, as frações são mais altas para os mesmos (D_0, n): a imagem borrada já não tem energia em alta frequência, então os passa-baixas quase não têm o que remover. Aplicar passa-baixas sobre a imagem borrada só a borra mais; nenhum deles reduz o *blur*.

Tentativa de atenuar o *blur*

Como o *blur* é uma perda de altas frequências, testamos realce por *unsharp masking*, $y = x_{\text{blur}} + k\,(1 - H_{\text{butt}}) * x_{\text{blur}}$, variando D_0 e o ganho k.



Figura 6: Imagem borrada e duas tentativas de realce.

Tabela 4: Erro RMS em relação à imagem original.

Imagem	**RMS**
Borrada	0,0944
Realçada, $D_0 = 10$, $n = 2$, $k = 0{,}25$	0,0939
Realçada, $D_0 = 5$, $n = 2$, $k = 1{,}0$	0,1116

O realce aumenta o contraste das bordas que sobreviveram e a imagem parece mais nítida, mas o erro em relação à original praticamente não melhora (0,0944 → 0,0939) e piora com ganho alto. A razão é que as máscaras são *isotrópicas* e o *blur* é direcional: o realce amplifica também as frequências verticais, que nunca foram degradadas, enquanto as horizontais anuladas pelos zeros do *sinc* não podem ser recuperadas por multiplicação no domínio da frequência — seriam necessárias divisão por H (filtro inverso) ou um filtro de Wiener, fora do escopo desta prática. A conclusão da parte final é negativa e esperada: filtragem passa-baixas/passa-altas isotrópica não desfaz um *blur* direcional.

Script (Octave)

```
close all
clear all
clc

pkg load signal
pkg load image

global VIS = 'off';

if ~exist('plots', 'dir'), mkdir('plots'); end
```

```
11
12  fig = 'lena_c.jpg';
13
14  filters = struct('name', {}, 'weights', {});
15
16  filters(1).name     = 'Lowpass';
17  filters(1).weights  = 1/9 * [  1  1  1;  1  1  1;  1  1  1 ];
18
19  filters(2).name     = 'Highpass';
20  filters(2).weights  = [ -1 -1 -1; -1  8 -1; -1 -1 -1 ];
21
22  filters(3).name     = 'Prewitt - Vertical';
23  filters(3).weights  = [ -1  0  1; -1  0  1; -1  0  1 ];
24
25  filters(4).name     = 'Sobel - Vertical';
26  filters(4).weights  = [ -1  0  1; -2  0  2; -1  0  1 ];
27
28  filters(5).name     = 'Prewitt - Horizontal';
29  filters(5).weights  = [ -1 -1 -1;  0  0  0;  1  1  1 ];
30
31  filters(6).name     = 'Sobel - Horizontal';
32  filters(6).weights  = [ -1 -2 -1;  0  0  0;  1  2  1 ];
33
34  filters(7).name     = 'Kirsch';
35  filters(7).weights  = [  5  5  5; -3  0 -3; -3 -3 -3 ];
36
37  filters(8).name     = 'Laplacian';
38  filters(8).weights  = [  1 -2  1; -2  4 -2;  1 -2  1 ];
39
40  % ------------------------------------------------------------------ helpers
41
42  function img = read_img(name)
43      root = './img/';
44      X = imread([root name]);
45      X = rgb2gray(X);
46      img = double(X)./255;
47  end
48
49  function save_fig(tag)
50      saveas(gcf, ['plots/' tag '.jpg']);
51  end
52
53  function show_spectrum(fig_id, img, tag)
54      global VIS
55      S = log(1 + abs(fftshift(fft2(img))));
56      figure(fig_id, 'visible', VIS);
57      colormap(gray(256));
58      imshow(mat2gray(S));
59      title(['Spectrum - ' tag]);
60      save_fig(['spectrum_' tag]);
61  end
62
63  function [X,Y] = plot_comparison(fig_id, name, spec)
64      global VIS
65      X = read_img(name);
66      Y = filter2(spec.weights, X);
67
68      figure(fig_id, 'visible', VIS);
69      subplot(1,2,1);
70      colormap(gray(256));
71      imshow(X);
72      title(name, 'interpreter', 'none');
73
74      subplot(1,2,2);
75      colormap(gray(256));
76      imshow(Y, []);
77      title(spec.name);
78
79      [~, base] = fileparts(name);
80      tag = strrep(spec.name, ' - ', '_');
81      saveas(gcf, ['plots/' base '_' tag '.jpg']);
82  end
83
```

```
function sweep(name, filters)
    for i = 1:numel(filters)
        plot_comparison(i, name, filters(i));
    end
end

% Masks below are centered, i.e. they live in the fftshift layout, so the
% spectrum has to be shifted before the product and unshifted afterwards.
function y = freq_filter(img, H)
    y = real(ifft2(ifftshift(fftshift(fft2(img)) .* H)));
end

function [Hi, Hb] = lp_masks(D, D0, n)
    Hi = double(D <= D0);
    Hb = 1 ./ (1 + (D ./ D0).^(2*n));
end

% Fraction of the spectral energy that survives the mask.
function e = energy_kept(img, H)
    F = fftshift(fft2(img));
    e = sum(sum(abs(F.*H).^2)) / sum(sum(abs(F).^2));
end

function compare_freq(fig_id, img, D, D0, n, tag)
    global VIS
    [Hi, Hb] = lp_masks(D, D0, n);
    yi  = freq_filter(img, Hi);
    yb  = freq_filter(img, Hb);
    yh  = freq_filter(img, 1 - Hb);

    figure(fig_id, 'visible', VIS);
    colormap(gray(256));
    subplot(2,2,1); imshow(img, []); title('Original');
    subplot(2,2,2); imshow(yi, []);  title(sprintf('Ideal LP  D0=%g', D0));
    subplot(2,2,3); imshow(yb, []);  title(sprintf('Butter LP  D0=%g  n=%d', D0, n));
    subplot(2,2,4); imshow(yh, []);  title(sprintf('Butter HP  D0=%g  n=%d', D0, n));
    save_fig(sprintf('%s_D0-%g_n-%d', tag, D0, n));

    printf('%-10s D0=%5.1f n=%d | energy kept: ideal %.4f  butter %.4f  butter HP %.4f\n'
        , ...
           tag, D0, n, energy_kept(img, Hi), energy_kept(img, Hb), energy_kept(img, 1-Hb)
               );
end

% ================================================================ Part 1

img = read_img(fig);

show_spectrum(100, img, 'original');
sweep(fig, filters);

% ================================================================ Part 2

filters(9).name    = 'Blurr - Horizontal';
filters(9).weights = 1/16 * ones(1,16);

filters(10).name    = 'Blurr - Vertical';
filters(10).weights = 1/16 * ones(16,1);

plot_comparison(9,  fig, filters(9));
plot_comparison(10, fig, filters(10));

[M,N] = size(img);
u = -M/2 : (M/2)-1;
v = -N/2 : (N/2)-1;
[V,U] = meshgrid(v, u);
D = sqrt(U.^2 + V.^2);

% (D0, n) pairs: the one required by the spec plus three variations.
configs = [ 50 2; 10 2; 120 2; 50 8 ];

printf('\n--- sharp image ---\n');
for k = 1:rows(configs)
```

```
    compare_freq(200+k, img, D, configs(k,1), configs(k,2), 'lena');
end

% Masks themselves, for reference.
[Hi, Hb] = lp_masks(D, 50, 2);
figure(300, 'visible', VIS);
colormap(gray(256));
subplot(1,3,1); imshow(Hi, []);     title('Ideal LP  D0=50');
subplot(1,3,2); imshow(Hb, []);     title('Butter LP  D0=50  n=2');
subplot(1,3,3); imshow(1 - Hb, []); title('Butter HP  D0=50  n=2');
save_fig('masks_D0-50_n-2');

% --- items 8 to 10: horizontally blurred image ---

blur = filter2(filters(9).weights, img);
show_spectrum(101, blur, 'blur');

printf('\n--- horizontally blurred image ---\n');
for k = 1:rows(configs)
    compare_freq(400+k, blur, D, configs(k,1), configs(k,2), 'blur');
end

% Unsharp masking: add back the high frequencies the blur attenuated.
figure(500, 'visible', VIS);
colormap(gray(256));
subplot(1,3,1); imshow(blur, []); title('Blurred');
printf('\nrms(blur - original) %.4f\n', sqrt(mean((blur(:)-img(:)).^2)));
sharp_cfg = [ 10 0.25; 5 1.0 ];
for i = 1:rows(sharp_cfg)
    D0 = sharp_cfg(i,1); k = sharp_cfg(i,2);
    [~, Hb] = lp_masks(D, D0, 2);
    sharp = blur + k * freq_filter(blur, 1 - Hb);
    subplot(1,3,i+1); imshow(sharp, []);
    title(sprintf('Unsharp D0=%g k=%g', D0, k), 'fontsize', 9);
    printf('unsharp D0=%5.1f n=2 k=%.2f | rms(sharp - original) %.4f\n', ...
           D0, k, sqrt(mean((sharp(:)-img(:)).^2)));
end
save_fig('blur_unsharp');
```

Listing 1: `main.m`